Anteater Chess

User Manual

Version 2.0
April 2026

Produced by: ChessEaters
EECS 22L Software Engineering Project in C Language
Henry Samueli School of Engineering
University of California, Irvine

Table of Contents

Glossary

The following chess and Anteater Chess terms are used throughout this manual.

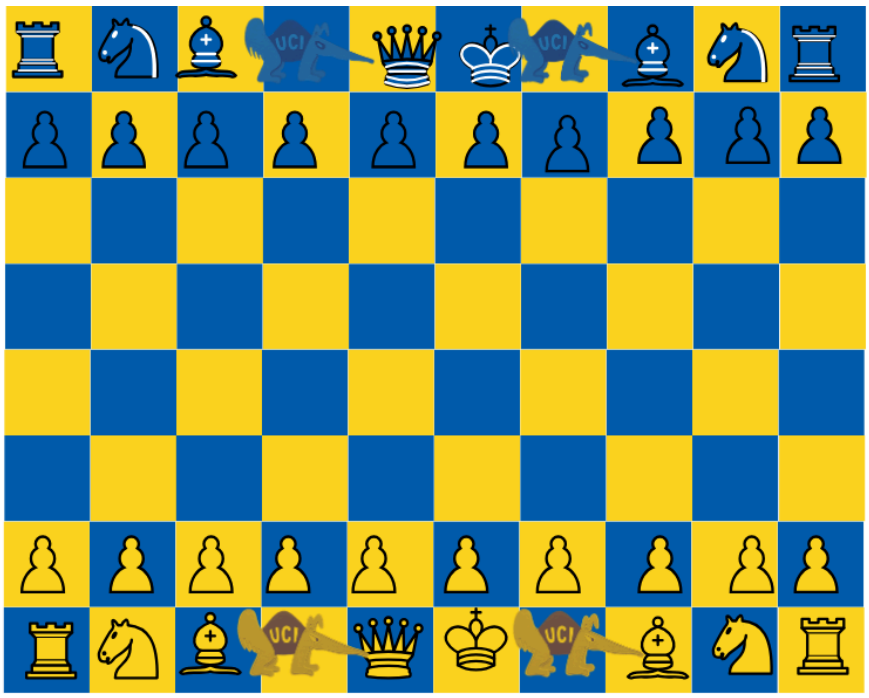
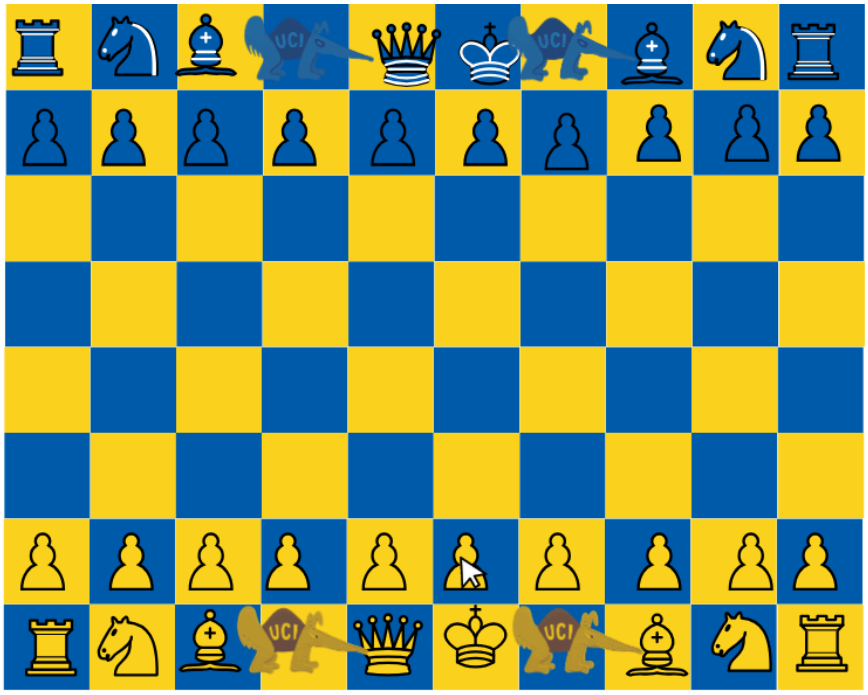
AI	Artificial Intelligence. The computer controlled opponent in Anteater Chess.
Ant	Alternative name for a pawn in Anteater Chess.
Anteater	A new piece unique to Anteater Chess. Moves one step in any direction and can chain capture adjacent pawns along the same rank or file.
Bishop	A piece that moves diagonally any number of squares.
Castling	A special simultaneous chess move between the king and rook, that will improve king safety and rook's activity. This move is available only when there are empty spaces between both, and both have not been moved yet.
Check	A state in which a king is under immediate threat of capture.
Checkmate	A state in which a king is in check and no legal move can remove the threat. The game ends.

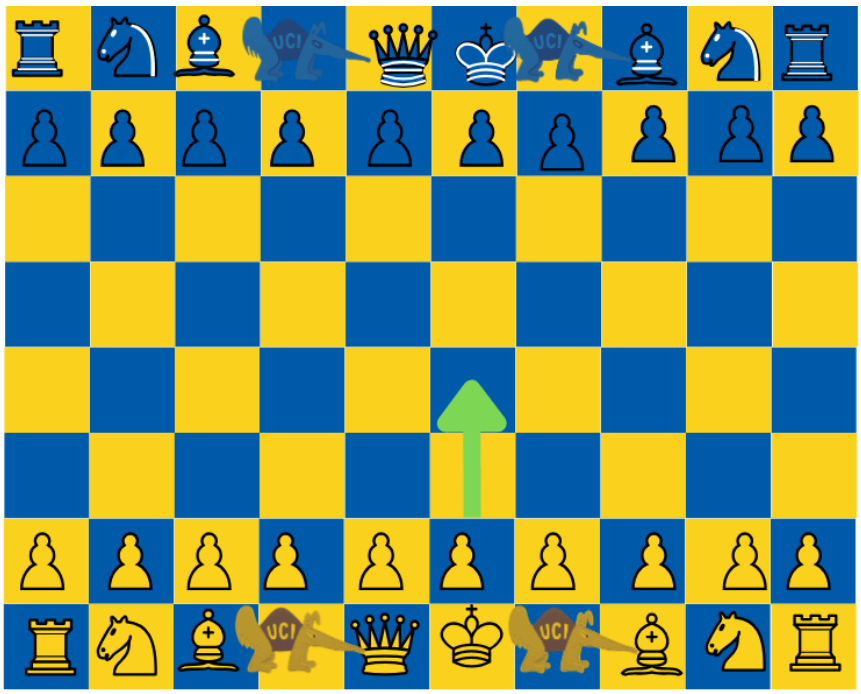
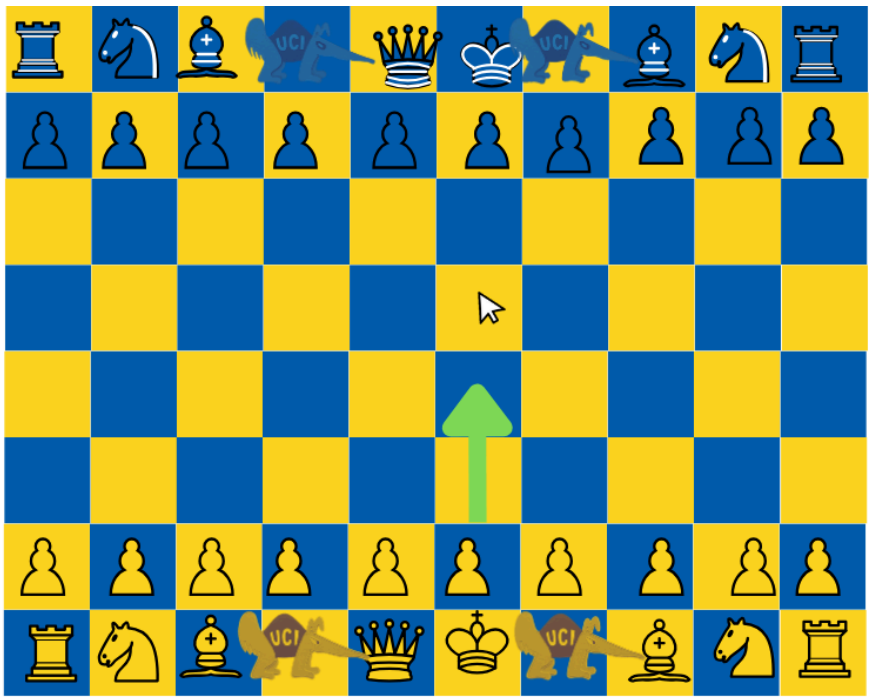
Chess Clock	A timer that tracks how much time each player has spent on their moves during the game. Selected time for each player, and the clock will start after a move has been made and stop once the player has completed their move.
En passant	A special pawn capture available immediately after an opponent's pawn advances two squares.
File	A column on the chess board, labeled A through J in Anteater Chess.
GUI	Graphical User Interface. The visual window based display of the game board.
Highlighted square	A square shown in a different color indicating a legal destination for the selected piece.
King	The most important piece; the game ends when it is checkmated.
Knight	A piece that moves in an L-shape: two squares in one direction, one square perpendicular.
Pawn	The most common piece; moves forward and captures diagonally. Referred to as Ant in Anteater Chess.
Promotion	When a pawn reaches the far end of the board, it is converted to another piece.
Queen	The most powerful piece; moves horizontally, vertically, or diagonally any number of squares.
Rank	A row on the chess board, numbered 1 through 8 in Anteater Chess.
Rook	A piece that moves horizontally or vertically any number of squares.
Stalemate	A draw condition in which the player to move has no legal moves but is not in check.
Undo	A function that reverses the player's most recent move.

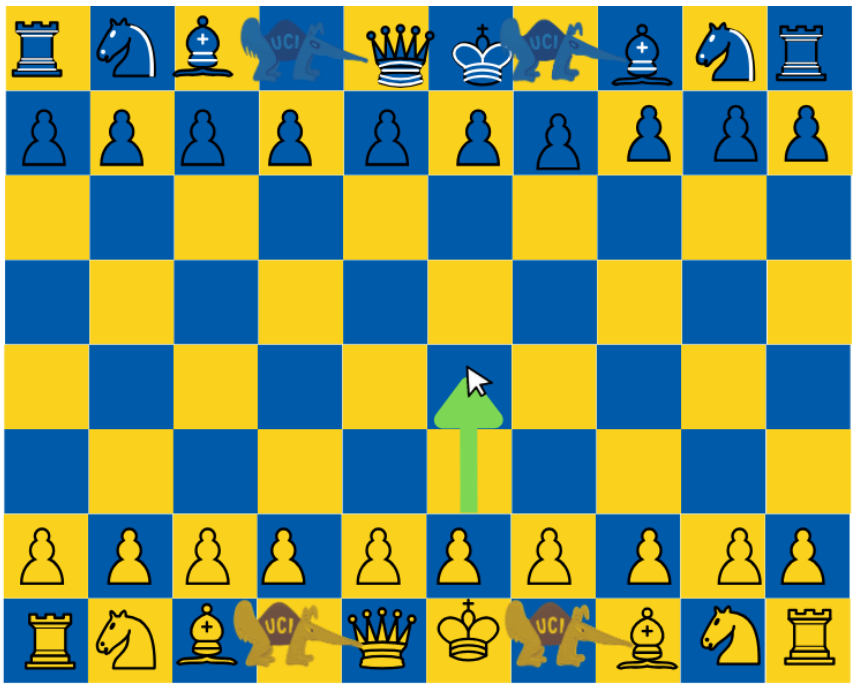
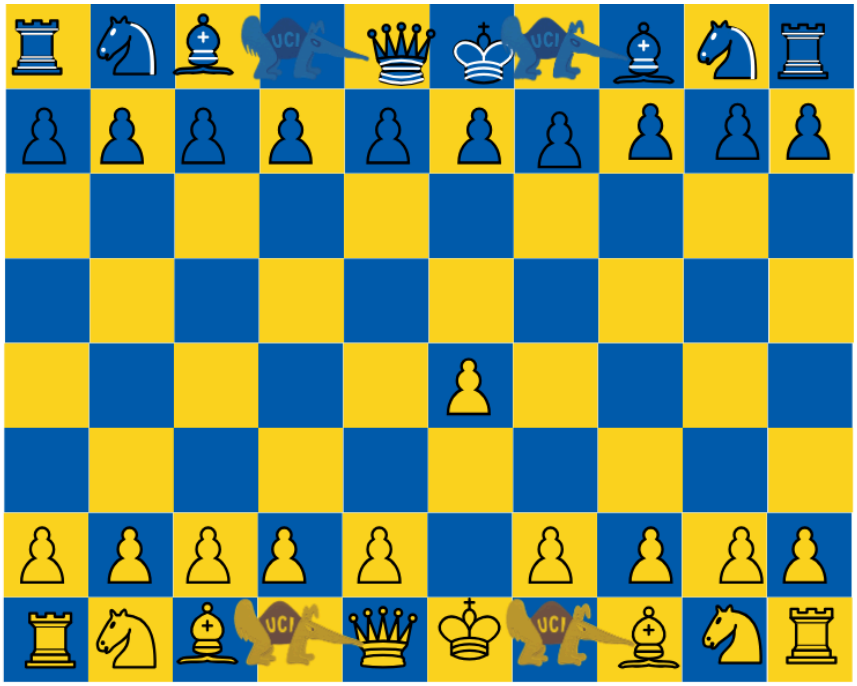
1. Computer Chess

1.1 Usage scenario

A user scenario is provided below:
- User attempts to make an illegal move
- When user chooses piece to move using cursor, projected paths will be highlighted on the board based on piece picked and placement
- If the user picks an illegal placement by cursor, the move will not be made and the user will not be able to place the piece, and will have to pick a legal move
- Once legal move is placed the move is made and the piece will appear at the selected placement

	User begins as yellow and can make the first move
	User selects pawn at F2 to move

	Selected pawn can move two spaces forward
	User selected three spaces forward, but can not make illegal move

	User then selects a legal move
	Legal move is accepted and the move is made by user

1.2 Goals

ChessEater's focus is to make chess accessible and enjoyable for all skill levels by simulating chess in a virtual environment. ChessEater is a software for virtual gameplay of Anteater chess, where the objective is to checkmate the opponent's king. The program will clearly display the winner and loser when a king is checkmated, time has finished, or a draw when a stalemate is drawn.

ChessEater's also aims to also educate users on the legal moves in Anteater chess. Our software is designed to assist the user by highlighting all legal moves for a selected piece, helping the player make a legal choice and preventing them from making an illegal choice. Our software is also designed to be easily interactive and accessible, including graphical and program features like difficulty selection, color selection, a chess clock and much more.

1.3 Features

The ChessEater software aims to simulate a well played game of Anteater chess. General features include: a piece selection and placement feature, with projected path placement, calibrated for each piece and current board placement; Standard chess rules, including piece capturing, special chess moves, color selections, checkmate and stalemates (draw). Additional functionalities include an automated chess clock, and additionally three selections for matchups. The ChessEater team has developed a strategic AI algorithm that any user may match up with, including a selection for the difficulty of the AI you may be matched with. This AI can use strategic move and piece capture to win the game, and can provide an interactive and challenging match for our users.

2. Installation

2.1 System requirements

The following are required to run Anteater chess.

- Operating system
- Linux (tested on UCI EECS servers: Bondi, Laguna, Crystalcove, Zuma, Balboa, Doheny)
- GCC compiler (version 9.0 or later)
- GNU Make
- OpenGL (for GUI display) pre installed on UCI servers
- X11 display (required for GUI display)
- Minimum 512 MB of RAM, 50 MB of free disk space

2.2 Setup and configuration

To install and run Anteater chess from the binary package, download the file and run the following commands.

- gtar xvzf Chess_V1.0.tar.gz
- chess/bin/chess

To build from source code, download the source package and run the following commands

- gtar xvzf Chess_V1.0_src.tar.gz
- cd chess
- make
- chess/bin/chess

2.3 Uninstalling

To remove Anteater chess, delete the extracted directory from your files with the following command

- rm -rf chess/

3. Chess Program Functions and Features.

3.1 Color Choice Option

This function allows the user to select their chess piece color they want to play as, whether it is black or white.

User Input:

- The user clicks on either a blue button or yellow button (TBD)

Program Output:

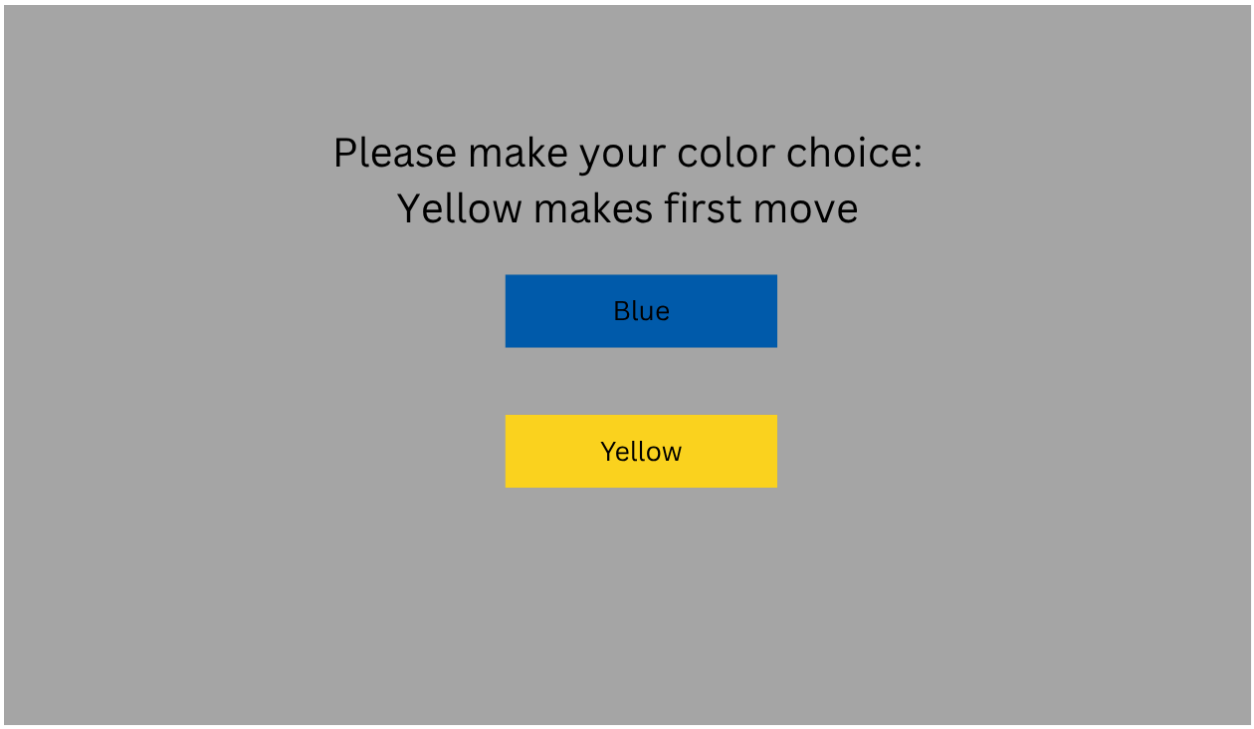
- The program detects the selected color.
- The player will only be able to interact with the selected color pieces.

Result:

- The user can select any of the selected color pieces and move them
- The user can interact with the opposing color by capturing
- Yellow will be able to make the first move, opposing color is blue

Error Handling:

- If the user clicks on the opposing color chess piece, no action will be taken



3.2 Projected Chess Piece Path

This function allows the user to view all valid moves for a selected chess piece before making a move.

User Input:

- The user clicks on a chess piece using the mouse.

Program Output:

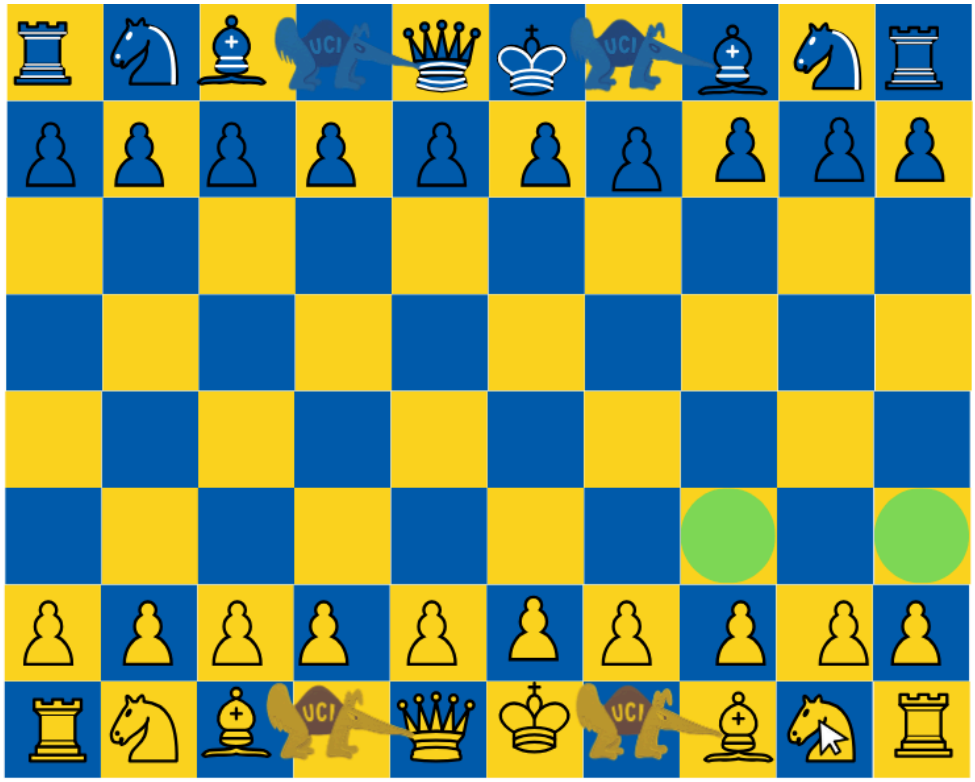
- The program detects the selected piece.
- All legal moves for that piece are calculated based on the current board state.
- Valid destination squares are highlighted on the board in a different color.

Result:

- The user can clearly see all possible legal moves and choose one of the highlighted squares.
- If no valid moves are available, no squares will be highlighted.

Error Handling:

- If the user clicks on an empty square or an opponent's piece, no moves are highlighted and no action is taken



3.3 Path Selection

The user will be able to select any highlighted path that is displayed when they click the chess piece that is the color they are playing.

User Input:

- The user would click on one of the highlighted paths

Program Output:

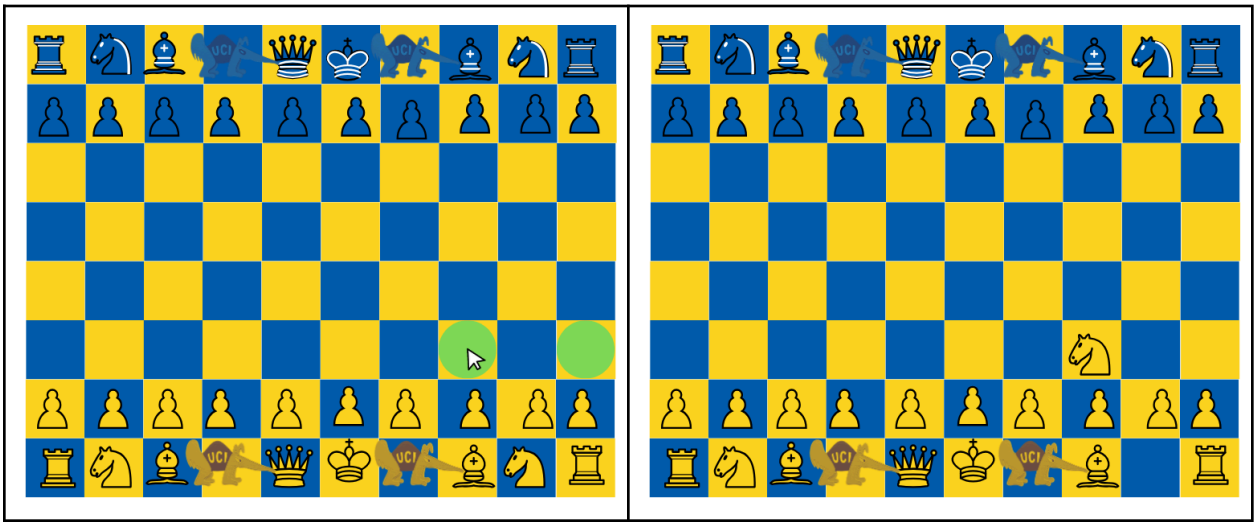
- The program detects the selected path
- The chess piece will move from its current position to the selected position
- The user ended its turn and allows the opponent to react to the displaced chess piece

Result:

- The chess piece will now be moved to its new position
- The user ends its turn and allows the opponent to make their move

Error Handling:

- If the user clicks on another chess piece after making a move or wishes to reclick the displaced chess piece again, no path will be displayed and no action will be taken.



3.4 Undo placement

The user may undo the most recent move they just placed, depending on their matchup.

User Input:

- The user will click the “undo last move” button.

Program Output:

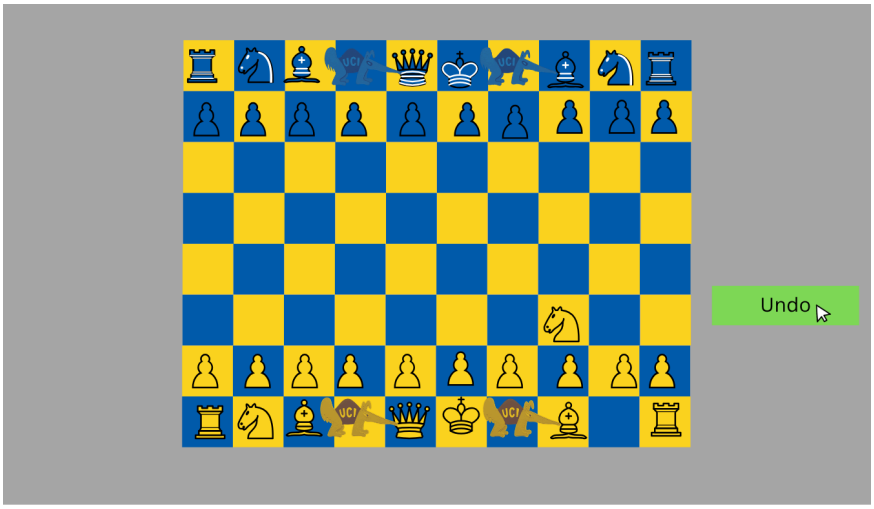
- If the user is against an AI, program will revert the most recently moved piece to the last board placement
    - If the opposing side had also moved, their most recently moved piece will also revert to previous placement
- If the user is playing against another user, the program will revert the most recently moved piece to the previous placement, only if the opposing user has not made their move yet
    - If the opposing user has already made their move, the undo placement will be invalid for the previous user

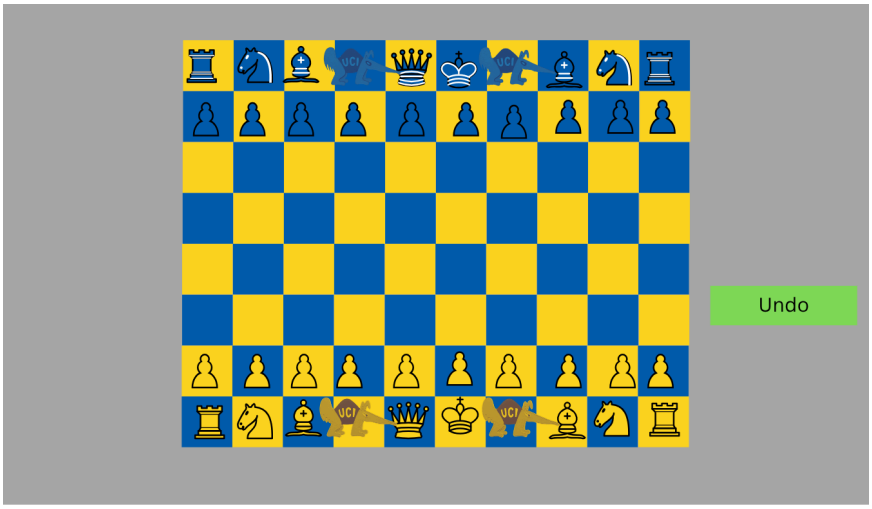
Result:

- After undo, it is now the user’s turn and user can now play with previous turn board placement

Error Handling:

- If the user clicks undo placement more than once, an error message will display informing the user that they may only go back one move.
- If the second user has placed their move, the undo placement button will only valid for them, and the first user will not have a valid undo button





3.5 Illegal Placement

This function prevents the user from making moves that violate the rules of the chess game.

User Input:

- The user selects a chess piece and attempts to move it to a square that is not highlighted as a valid move.

Program Behavior / Output:

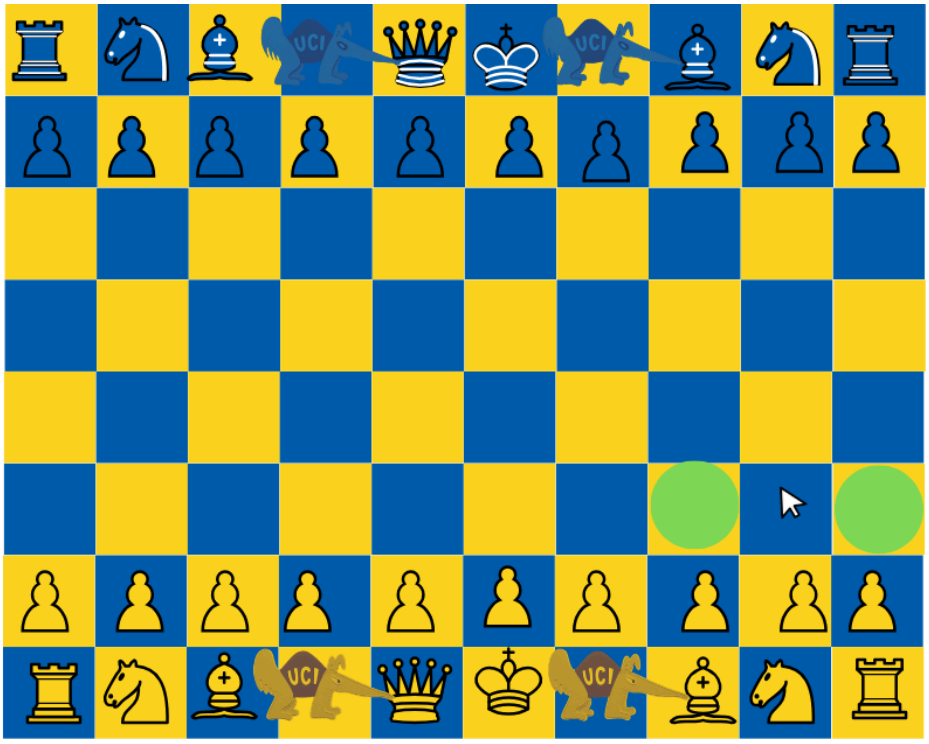
- The program checks whether that selected move is legal.
- If the move is illegal, the program rejects the action.
- The board state remains unchanged, and the piece does not move.

Result:

- The user must select a valid highlighted square to complete their move.
- No visual or positional changes occur for illegal moves.

Error Handling:

- Invalid moves are ignored without crashing or altering the game state.
- The user can immediately attempt another move.



3.6 Run Out Time

This function ends the game when a user's time expires.

User Input:

- No direct input is required. This occurs automatically when a user's timer reaches zero.

Program Behavior / Output:

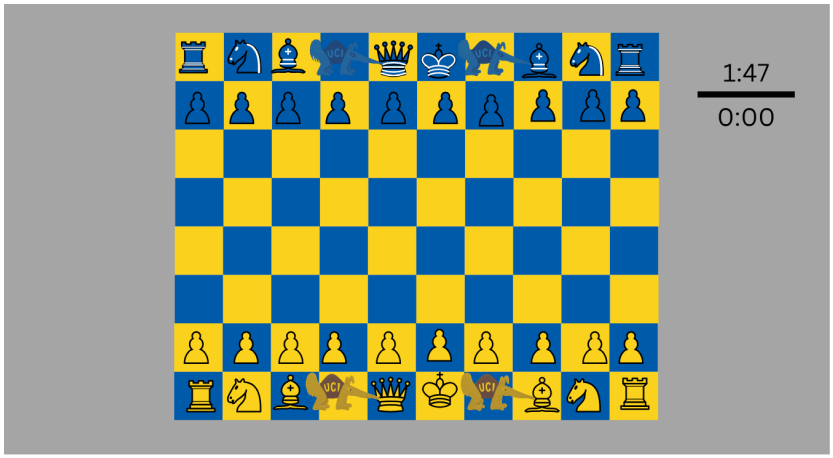
- The program continuously tracks each player's remaining time.
- When a player's timer reaches zero, the game immediately ends.
- A losing message is displayed for the player who ran out of time.

Result:

- The opponent is declared the winner.
- No further moves can be made.

Error Handling:

- The timer cannot go below zero and stops automatically when the time expires.
- As soon as timer expires user's cursor will no longer register



3.7 Victory Message

When a player has checkmated their opponent, victory message will display for winner

User Input:

- When one of the players has reached a checkmate, based on color, the winner will be automatically assigned

Program Behavior / Output:

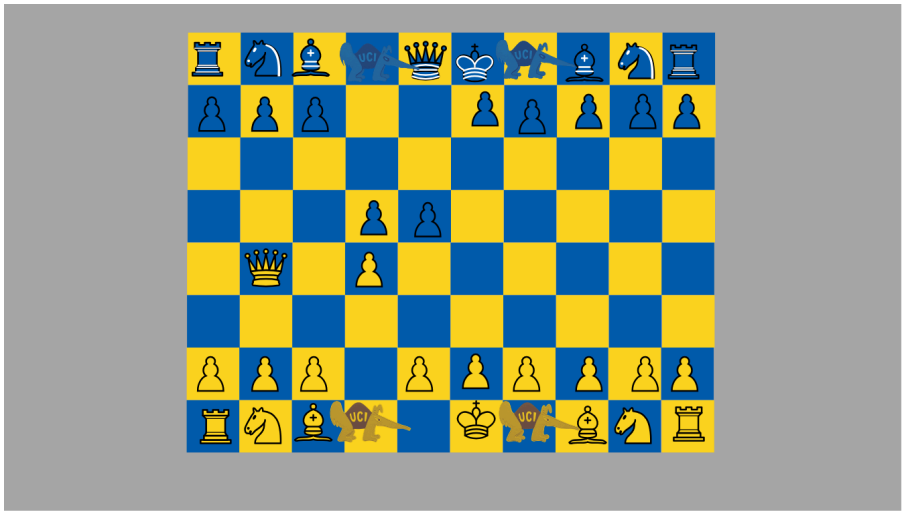
- The program's overall algorithm will continuously be checking for a checkmate for either side's king
- As soon as one side has placed the opposing side in checkmate, a victory message will display on their side of the screen
    - On the opposing side a losing message will display

Result:

- The player is declared the winner.
- No further moves can be made.

Error Handling:

- As soon as the winning message is displayed, neither player can make any further moves
- The uppermost algorithm will be checking against the king for any checkmated moves



3.8 Stalemate

When a player has not been placed in checkmate, but can not make any further moves, a stalemate will be declared and the message will display

User Input:

- Occurs only when the player currently playing is not in a checkmate, but has no further legal moves left

Program Behavior / Output:

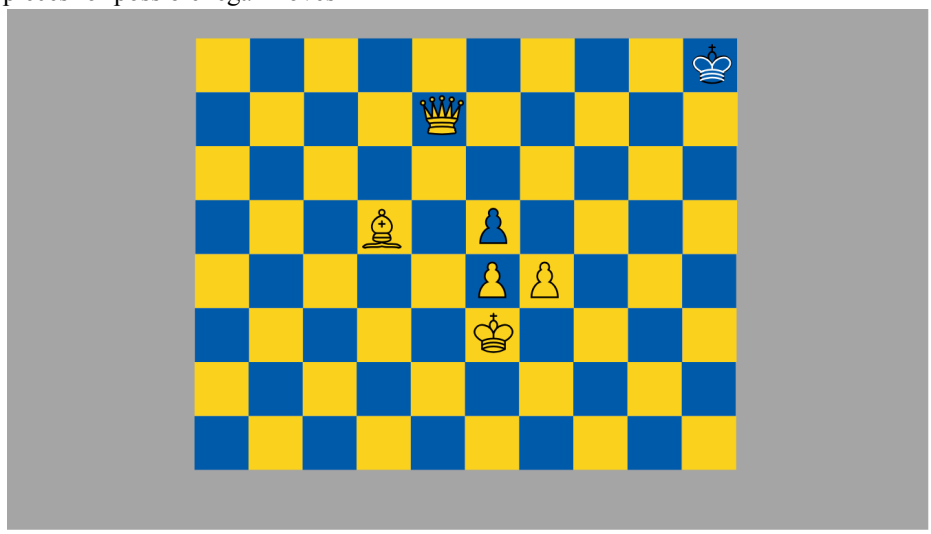
- The program will continuously be checking for a stalemate for the current player, by checking all their remaining pieces' possible moves, and the if their king is in checkmate
- When a stalemate has been declared that side will have a message displayed informing them they have no possible moves left, and that the score is a draw

Result:

- The player will be have their message displayed, and that a draw has been made
- No further moves can be made.

Error Handling:

- As soon as the stalemate message is displayed, neither player can make any further moves
- The algorithm will be checking against the king for any checkmated moves and against remaining pieces for possible legal moves



3.9 Difficulty selection

When playing against a computer opponent, the user will be given 3 difficulty levels: easy, medium, hard and may choose which level they want to play against

User Input:

- After choosing their opponent to be the computer, the user will select difficulty selection button for easy, medium or hard

Program Behavior / Output:

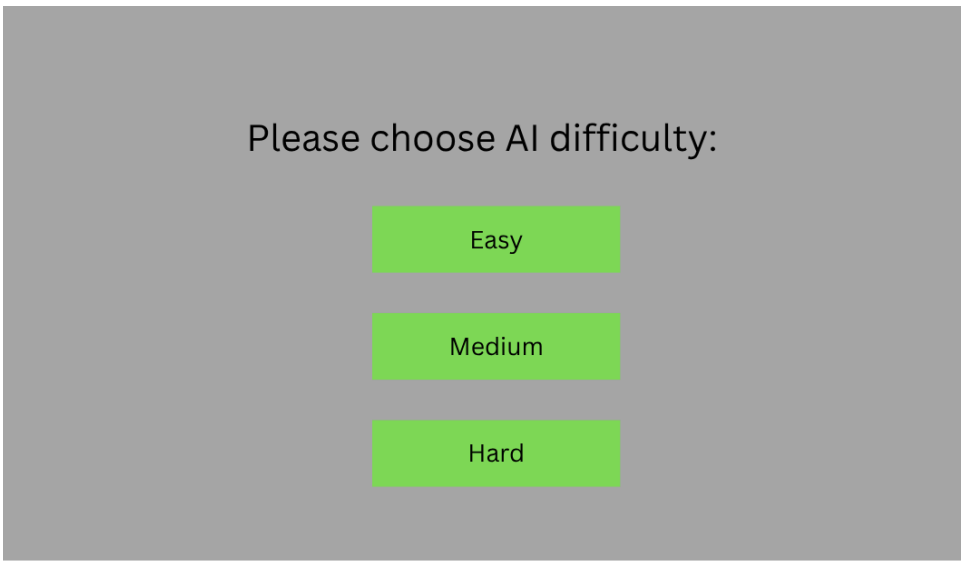
- After selection has been made the program's playing mode will play using the selected level of competitiveness
- For each level, program will use AI algorithms that will be making basic or more intelligent move placements during the chess session, based on level

Result:

- The player will be able to play against an easier or more difficult computer generated player
- The computer will be generating moves assessed on a lower or higher level strategy

Error Handling:

- After selection has been placed user will have to finish game in entirety under the selected difficulty level
- Based on selection, the program will be using the algorithm that will generate less or more strategic gameplay



3.10 Chess Clock

The functionality provides the timing convention for both user and opponent to make their moves under the set time limit

User Input:

- The user and opponent will select the agreed upon time given to each of the players at the start of the game.

Program Output:

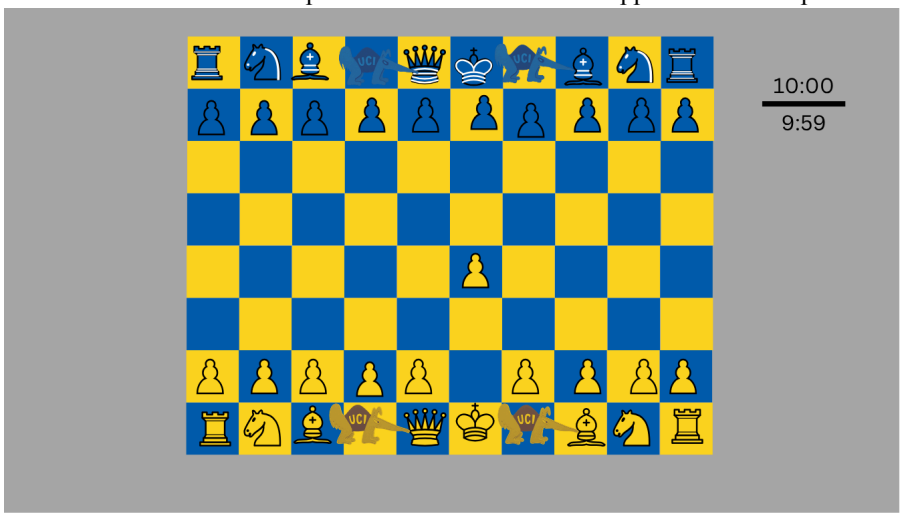
- The clock timer counts down during a player's move, and pauses when their move is completed.
- The program displays two timers on the screen for each player, and will be running an internal clock to count down seconds

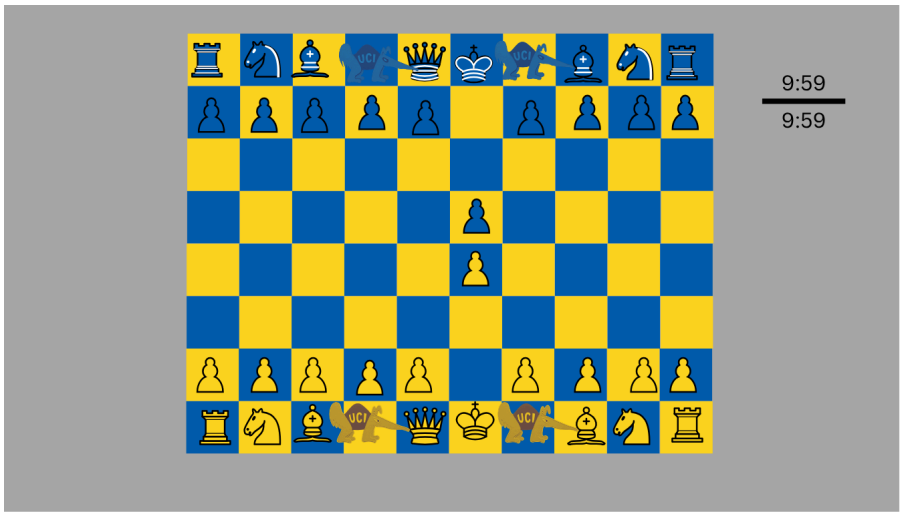
Result:

- The program automatically switches between timers after each move.
- The remaining time for each player is continuously updated and displayed.

Error Handling:

- The timer cannot decrease below zero and is controlled automatically by the program.
- Timers will also stop and end after each user or opponent has completed their move





3.11 Matchups

The user will be able to select the different matchups available to play against as their opponent

- The program will start the game based on the selected matchup.

User Input:

- The player matchups will have three options: Player vs. Player; Player vs. AI, or AI vs. AI
- The user may select either option as a button at the start of their game

Program Output:

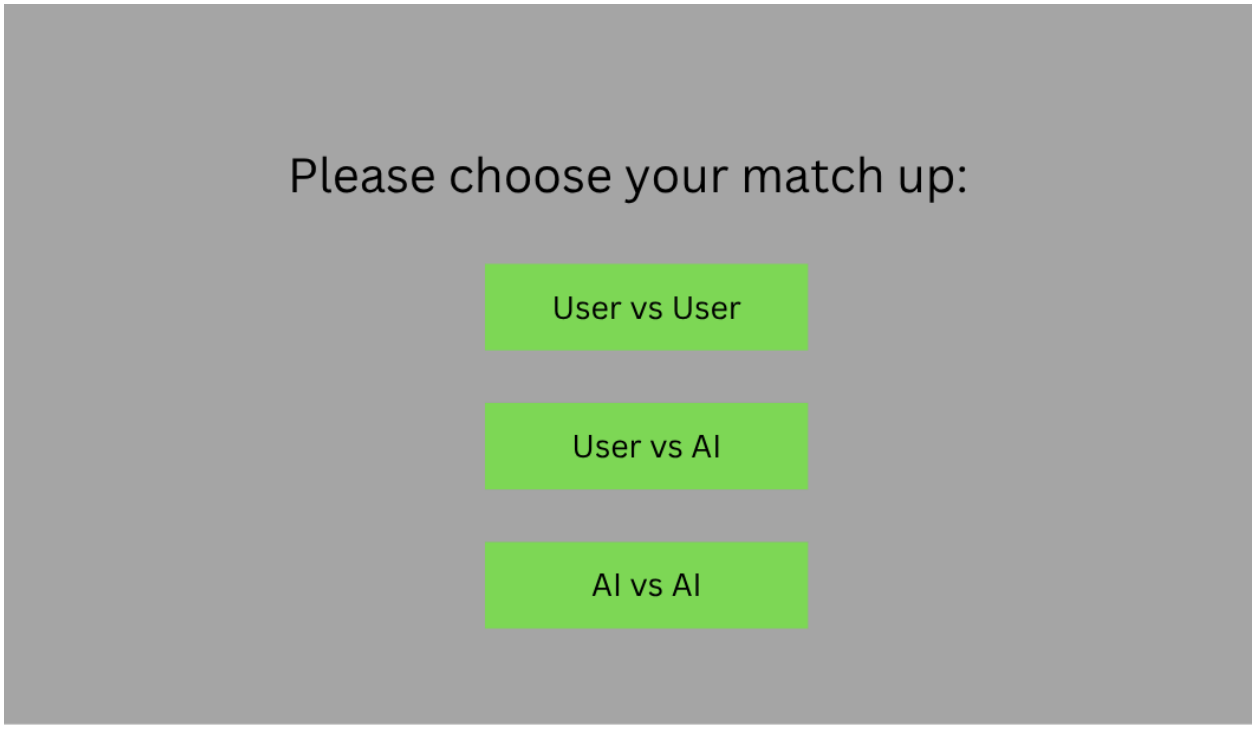
- The clock timer counts down during a player's move, and pauses when their move is completed.
- The program displays two timers on the screen for each player, and will be running an internal clock to count down seconds

Result:

- The program automatically switches between timers after each move.
- The remaining time for each player is continuously updated and displayed.

Error Handling:

- The timer cannot decrease below zero and is controlled automatically by the program.
- Timers will also stop and end after each user or opponent has completed their move



Back matter

COPYRIGHT

© 2026 ChessEaters, University of California, Irvine. All rights reserved. This software was developed as a course project for EECS 22L at UCI. Redistribution and use in source and binary forms, with or without modification, are permitted for educational purposes only.

TROUBLESHOOTING AND ERROR MESSAGES

Error / Condition	Behavior and Resolution
Invalid move	If a user attempts to click on a part of the board that is not highlighted as a valid move, there will be no response from the computer and the clock timer continues on their turn.
Timer expired	When a player's timer reaches zero, the game ends and a losing message displays that the user lost because of time.
No moves left	If there are no legal moves left for the player to choose, the game will end automatically.

INDEX

CONTACT INFORMATION

For questions, feedback, or issues regarding this software, please contact one of the team members below:
clairln1@uci.edu
csharrin@uci.edu
jordall2@uci.edu
tarapai.04@gmail.com
yamontan@uci.edu
aureaj@uci.edu

LEGAL, LICENSE, AND DISCLAIMER OF WARRANTY

This software is developed for educational and testing purposes only and is not intended for commercial use. Developers assume no liability for errors, misuse, or damages arising from the use of this program.

REFERENCES

[1] *Chess.com.* Rules of Chess. https://www.chess.com/learn-how-to-play-chess. Accessed April 2026.

[2] *UCI EECS 22L Course Staff.* Anteater Chess Project Specification. University of California, Irvine, 2026.